Assignment on Change Enablement

1. Introduction & Context

This report details how ITIL v4 Change Enablement is integrated into the software maintenance workflow for the RepairCafé Ticketing System.
The RepairCafé project involved building a web application to track repair items, client details, and volunteer assignments. The initial software development lifecycle was structured around three key phases:

1. Initial Analysis: A meeting was held with the client to analyze the ticketing problems, define requirements, and understand the workflow of a typical RepairCafé day.
2. Proof of Concept (PoC): A prototype was developed and presented to the client. Feedback was gathered informally in loosely scribbled notes during the meetings
3. Final Delivery: The final product was deployed and delivered to the client. Upon delivery, the client provided further feedback regarding minor bugs and desired adjustments.

Throughout this process, Git was used as the version control system. However, no automated testing, linting, or Quality Assurance (QA) pipelines were implemented. I relied solely on manual validation. This report evaluates the current state of the change enablement workflow and defines a structured, ITIL-compliant Change Enablement framework to ensure service quality, stability, and controlled delivery of system updates.

2. Current State Analysis and Gaps

Under ITIL v4, the purpose of Change Enablement is to maximize the number of successful service and product changes by ensuring that risks have been properly assessed, authorizing changes to proceed, and managing the change schedule (three severity levels).

2.1 Strengths in the Current Workflow

• Version Control (Git): The consistent use of Git provided an audit trail of changes through commits. This supports ITIL's requirements for traceability and rollback capability.
• Early Feedback (PoC): Demonstrating a PoC to the customer helped validate requirements early, reducing the risk of a (complete) mismatch between the delivered application and the client's needs.

2.2 Critical Gaps and Risks

• Lack of QA & Automated Testing: Relying entirely on manual testing is a high-risk approach. In Next.js applications, server-side logic (Prisma queries, Server Actions) and client-side components are closely coupled. Manual verification cannot guarantee that a new feature or database schema migration does not introduce regressions in unrelated areas.
• Informal Change Requests and Documentation: Gathering feedback in loosely scribbled notes during PoC reviews introduced ambiguity, leading to change requests not being implemented properly. Key requirements and bug reports got easily lost / were misinterpreted, leading to poor implementation.
• Absence of Release Validation: The fact that the customer still had outstanding feedback on the final product indicates that the changes were not fully verified and validated against clear acceptance criteria prior to production deployment.

3. Proposed ITIL v4 Change Enablement Framework

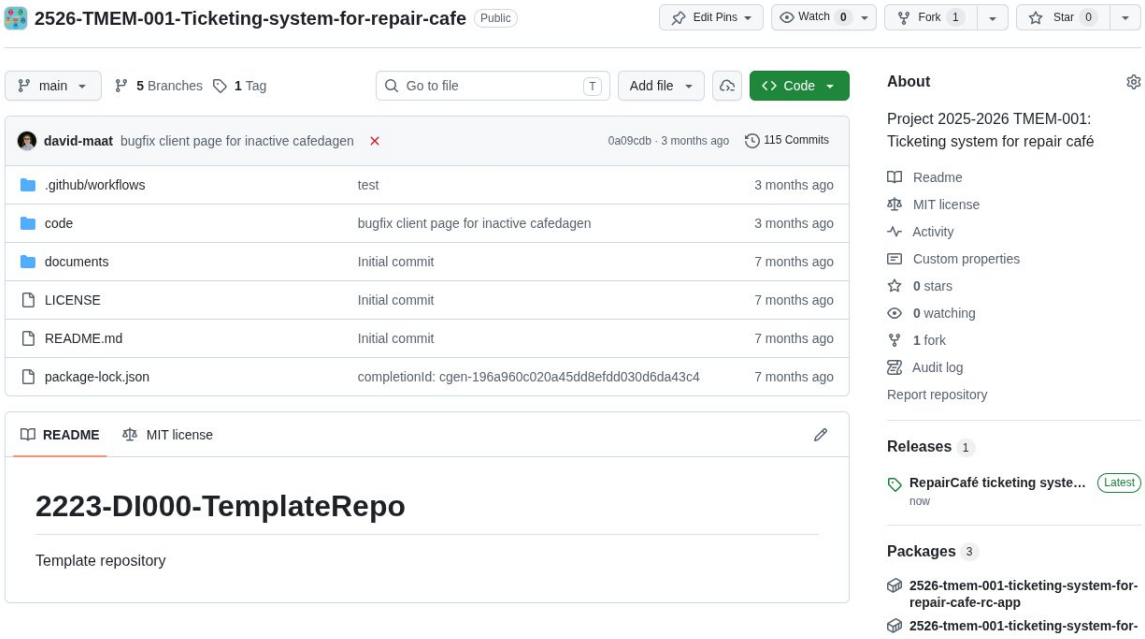
To transition the RepairCafé project into a stable, controlled software maintenance workflow, a structured Change Enablement framework must be established. This framework consists of change classification, a formalized change lifecycle, and automated QA gates.
To prevent regressions and ensure customer satisfaction, every change must follow a defined lifecycle:

1. Request and Logging (RFC): All client feedback and new requirements must be formally registered as a Request for Change (RFC) in a tracking tool (GitHub Issues or JIRA), rather than scribbled notes. Each RFC must outline the goal and impact.
2. Risk & Impact Assessment: I assesses the change against potential risks, such as breaking changes and documents how to handle these.
3. Build & Automated QA Gates:
   - Unit & Integration Testing: Implement automated tests using Vitest for business logic.
   - Continuous Integration: Set up a GitHub Actions workflow that executes tests and linters automatically on every pull request.
4. Staging & User Acceptance Testing: Deploy the change to a staging environment (a replica of production running on a staging database). The client performs UAT on the staging server to verify features and provide feedback (through RFC) before the production release.
5. Authorization & Release: Once a change is approved, the change is authorized for release. Release and deployment are automated via CD (Continuous Delivery).
6. Post-Implementation Review (PIR): After deployment, a brief review is conducted with the client to verify that the change solved the initial problem, caused no incidents, and is fully operational.

4. Conclusion

By replacing the ad-hoc testing and informal feedback loops of the RepairCafé project with an ITIL-aligned Change Enablement framework, service quality is significantly enhanced. The integration of automated QA gates (CI/CD), clear change classification, and staging-based user acceptance testing minimizes deployment risks, guarantees stability, and ensures that the final product meets the client's expectations without outstanding issues.

Assignment on Release Management



2526-TMEM-001-Ticketing-system-for-repair-cafe Public

Edit Pins | Watch 0 | Fork 1 | Star 0

main | 5 Branches | 1 Tag | Go to file T | Add file | Code

david-maat bugfix client page for inactive cafedagen — 0a09cdb · 3 months ago — 115 Commits

.github/workflows	test	3 months ago
code	bugfix client page for inactive cafedagen	3 months ago
documents	Initial commit	7 months ago
LICENSE	Initial commit	7 months ago
README.md	Initial commit	7 months ago
package-lock.json	completionId: cgen-196a960c020a45dd8efdd030d6da43c4	7 months ago

README | MIT license

2223-DI000-TemplateRepo

Template repository

About

Project 2025-2026 TMEM-001: Ticketing system for repair café

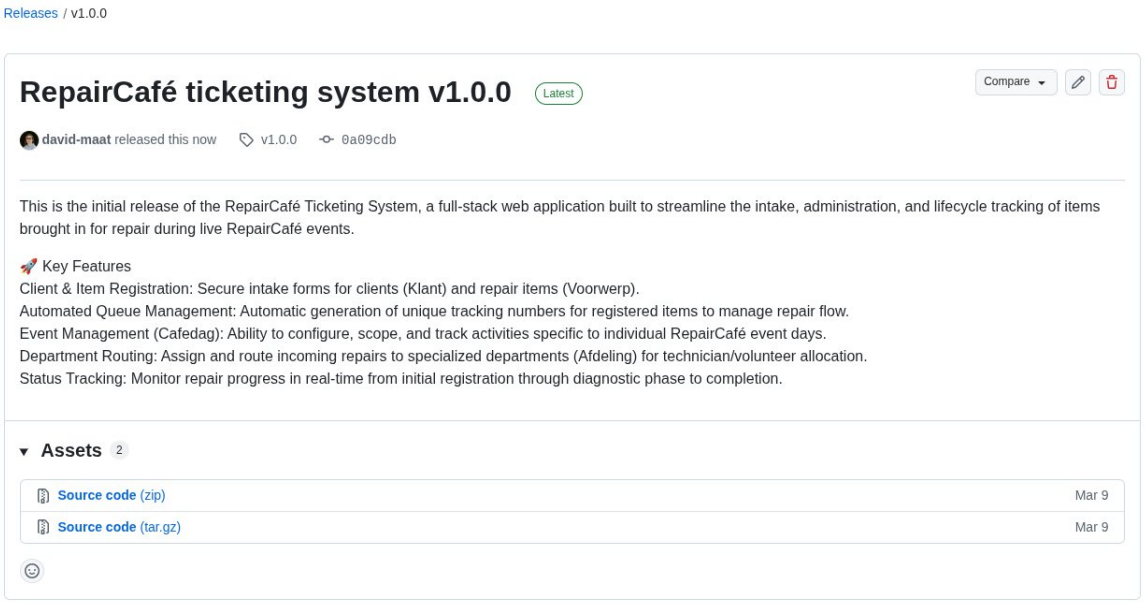
Readme
MIT license
Activity
Custom properties
0 stars
0 watching
1 fork
Audit log
Report repository

Releases 1

RepairCafé ticketing syste… Latest
now

Packages 3

2526-tmem-001-ticketing-system-for-repair-cafe-rc-app
2526-tmem-001-ticketing-system-for-



Releases / v1.0.0

RepairCafé ticketing system v1.0.0 Latest

Compare

david-maat released this now — v1.0.0 — 0a09cdb

This is the initial release of the RepairCafé Ticketing System, a full-stack web application built to streamline the intake, administration, and lifecycle tracking of items brought in for repair during live RepairCafé events.

🚀 Key Features
Client & Item Registration: Secure intake forms for clients (Klant) and repair items (Voorwerp).
Automated Queue Management: Automatic generation of unique tracking numbers for registered items to manage repair flow.
Event Management (Cafedag): Ability to configure, scope, and track activities specific to individual RepairCafé event days.
Department Routing: Assign and route incoming repairs to specialized departments (Afdeling) for technician/volunteer allocation.
Status Tracking: Monitor repair progress in real-time from initial registration through diagnostic phase to completion.

Assets 2

Source code (zip)	Mar 9
Source code (tar.gz)	Mar 9